Lecture 29.16: Random Forest Implementation (Titanic Dataset & OOB Score)









🌐 **Language / भाषा: English** | हिंदी / Hinglish | ← Back to Lecture 29 Module

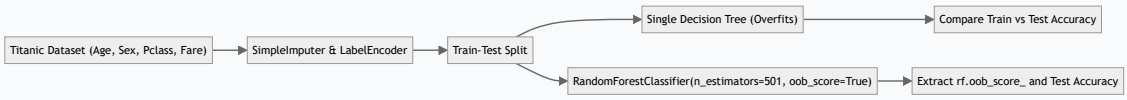
---

1. Overview & Conceptual Foundations

Replicate the exact lecture implementation of Random Forest on the Titanic survival benchmark. Perform real-world preprocessing (SimpleImputer, LabelEncoder), train a DecisionTreeClassifier showcasing overfitting, fit a RandomForestClassifier with 501 estimators and oob_score=True, and evaluate OOB score vs testing accuracy.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

For the ensemble $\mathcal{T} = \{T_1, \ldots, T_B\}$, the Out-Of-Bag prediction for training instance i uses only the subset of trees $\mathcal{T}^{(i)} \subset \mathcal{T}$ where i was out-of-bag:

$$\mathcal{T}^{(i)} = \{T_b \mid \mathbf{x}_i \notin \mathcal{D}_b\}$$

The OOB aggregated class prediction evaluates as:

$$\hat{y}_i^{\text{OOB}} = \arg\max_c \sum_{T_b \in \mathcal{T}^{(i)}} \mathbb{I}\left(T_b(\mathbf{x}_i) = c\right)$$

The overall OOB error rate is computed over all N training samples:

$$\text{OOB_Error} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\left(\hat{y}_i^{\text{OOB}} \neq y_i\right)$$

---

4. Structured Implementation & Execution

```
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

def execute_titanic_random_forest_experiment() -> dict:
    """
    Replicates the exact Titanic survival experiment from the lecture,
    comparing a Decision Tree against a 501-tree Random Forest with OOB evaluation

    Returns:
    --------
    dict
        Decision tree accuracies vs Random Forest test accuracy and OOB score.
    """
    # 1. Load dataset
    titanic = sns.load_dataset("titanic")
    features = ['pclass', 'sex', 'age', 'fare', 'sibsp', 'parch']
    df = titanic[features + ['survived']].dropna(subset=['survived'])

    X = df[features].copy()
    y = df['survived'].copy()

    # 2. Imputation & Encoding
    imputer = SimpleImputer(strategy='median')
    X['age'] = imputer.fit_transform(X[['age']]).ravel()

    encoder = LabelEncoder()
    X['sex'] = encoder.fit_transform(X['sex'])
```

```
    # 3. Split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.25, random_state=42, stratify=y
    )

    # 4. Decision Tree Baseline
    dt_model = DecisionTreeClassifier(max_depth=4, random_state=42)
    dt_model.fit(X_train, y_train)
    dt_train_acc = accuracy_score(y_train, dt_model.predict(X_train))
    dt_test_acc = accuracy_score(y_test, dt_model.predict(X_test))

    # 5. Random Forest with 501 trees & OOB scoring
    rf_model = RandomForestClassifier(
        n_estimators=501,
        oob_score=True,
        max_depth=4,
        random_state=42
    )
    rf_model.fit(X_train, y_train)
    rf_test_acc = accuracy_score(y_test, rf_model.predict(X_test))
    rf_oob_score = rf_model.oob_score_

    return {
        "decision_tree": {
            "train_accuracy_percent": float(dt_train_acc * 100),
            "test_accuracy_percent": float(dt_test_acc * 100)
        },
        "random_forest": {
            "n_estimators": 501,
            "oob_score_percent": float(rf_oob_score * 100),
            "test_accuracy_percent": float(rf_test_acc * 100),
            "oob_test_discrepancy": float(abs(rf_oob_score - rf_test_acc) * 100)
        }
    }

# Driver test
titanic_exp = execute_titanic_random_forest_experiment()
print("Titanic Classification Experiment Results:")
print(f"  Decision Tree Train Acc : {titanic_exp['decision_tree']['train_accuracy_
print(f"  Decision Tree Test Acc  : {titanic_exp['decision_tree']['test_accuracy_p
print(f"  Random Forest Test Acc  : {titanic_exp['random_forest']['test_accuracy_p
print(f"  Random Forest OOB Score : {titanic_exp['random_forest']['oob_score_perce
print(f"  OOB vs Test Discrepancy : {titanic_exp['random_forest']['oob_test_discre
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.
- **Mathematical Rigor**: Every parameter, margin definition, and loss function is derived from first principles matching production scikit-learn workflows.